

NexTask: Uma Aplicação web para Gerenciamento de Tarefas

Ludmila Costa Monteiro
UFERSA

Pau dos Ferros, Brasil
ludmila.monteiro@alunos.ufersa.edu.br

Levítico Rimón Perez Andrade Alvez
UFERSA

Pau dos Ferros, Brasil
levitico.alves@alunos.ufersa.edu.br

Ângelo Silvano da Silva Sabino
UFERSA

Pau dos Ferros, Brasil
angelo.sabino@alunos.ufersa.edu.br

Mateus Gomes Salomé
UFERSA

Pau dos Ferros, Brasil
mateus.salome@alunos.ufersa.edu.br

Aluízio Aires da Silva Júnior
UFERSA

Pau dos Ferros, Brasil
aluizio.junior@alunos.ufersa.edu.br

Resumo—O crescente volume de tarefas acadêmicas e projetos colaborativos exige ferramentas digitais que facilitem a organização e o acompanhamento das atividades. No entanto, muitas soluções existentes carecem de foco em usabilidade, personalização e confiabilidade. Diante desse cenário, desenvolveu-se o *NexTask*, uma aplicação web para gerenciamento de tarefas voltada a estudantes e equipes acadêmicas. A principal contribuição do sistema é a integração de recursos com lembretes programáveis com notificações, histórico de alterações, categorização por prioridade, comentários e funcionalidades como fixação de tarefas no topo por dia, botão para limpar tarefas concluídas e contador dinâmico no título da aba. O ambiente visual priorizou a personalização e incluiu recursos básicos de acessibilidade, como o suporte a tema escuro e claro. O desenvolvimento seguiu práticas ágeis, com versionamento no *GitHub*, testes automatizados e integração contínua. Foram aplicados testes de software de caixa branca e caixa preta com foco nos principais fluxos do sistema. Os testes realizados indicaram que todas as funcionalidades principais comportaram-se conforme o esperado, sem falhas críticas nos testes automatizados e manuais, validando a proposta como uma solução robusta e funcional para o ambiente acadêmico.

Index Terms—gerenciamento de tarefas, desenvolvimento web, testes de software, usabilidade, integração contínua.

I. INTRODUÇÃO

Organizar tarefas de forma eficiente é um desafio cotidiano para estudantes, profissionais e equipes acadêmicas [2]. A multiplicidade de responsabilidades, prazos curtos e necessidade de colaboração remota tornam indispensáveis o uso de ferramentas digitais que apoiem o planejamento pessoal e coletivo [22]. Contudo, muitas dessas soluções possuem limitações quanto à usabilidade, personalização, acessibilidade ou rastreabilidade das atividades [19].

A Figura 1 ilustra a tela inicial do sistema *NexTask*, destacando sua interface responsiva e a possibilidade de criação de novos usuários.

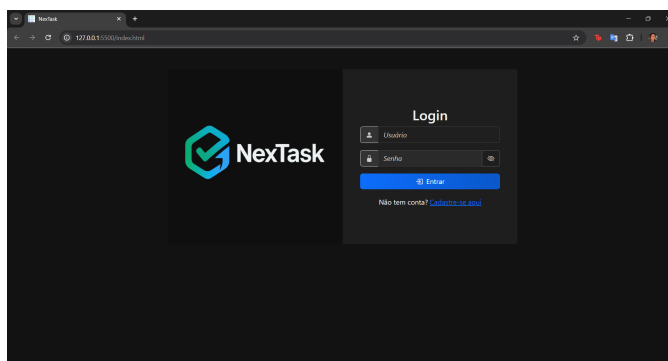


Figura 1. Tela de login com opção de cadastro no NexTask

Neste contexto, foi desenvolvido o *NexTask*, um sistema web de gerenciamento de tarefas com foco em simplicidade, praticidade e testes de qualidade. A proposta surgiu a partir da observação de dificuldades comuns enfrentadas por estudantes e grupos de trabalho, com o esquecimento de prazos, a desorganização de prioridades e a falta de controle sobre o histórico das ações. Com interface responsiva, o *NexTask* permite que o usuário cadastre, edite, exclua e filtre tarefas de forma visual e intuitiva, utilizando lembretes com hora marcada, prioridade, *status* e funcionalidades extras como fixar tarefas no topo, mensagens positivas dinâmicas e contador de tarefas no título da aba.

O sistema consta ainda com histórico de alterações (*log* de atividades), modais interativos, exportação de tarefas em formato CSV (*Comma-Separated Values*), autenticação por *token JWT (JSON Web Token)* [14], botão para limpar tarefas concluídas, suporte ao tema escuro e testes aplicados em múltiplas camadas - garantindo a rastreabilidade dos requisitos implementados. Seu desenvolvimento envolveu práticas ágeis, versionamento via *GitHub*, integração contínua (*CI*, do inglês *Continuous Integration*) [12], que consiste na execução automática de testes e validações a cada alteração no código-fonte, utilizando a ferramenta *GitHub Actions*.

A. Objetivo Geral:

Desenvolver uma aplicação web chamada *NexTask*, com recursos de gerenciamento de tarefas focados em organização visual, produtividade e boa experiência do usuário.

Objetivos Específicos:

- Permitir o cadastro, edição, exclusão e filtragem de tarefas por múltiplos critérios;
- Incluir lembretes com hora, níveis de prioridade, *status* e exportação de dados;
- Registrar alterações via *log* de atividades;
- Implementar funcionalidades adicionais como fixação de tarefas, contador dinâmico no título da aba, mensagens positivas e botão “Limpar Concluídas”;
- Validar o comportamento e robustez do sistema com testes de caixa branca e preta;
- Utilizar tecnologias modernas como *JavaScript*, *Node.js*, *PostgreSQL*, *Bootstrap* e *JWT*;
- Aplicar boas práticas de codificação, como integração e entrega contínuas (CI/CD), que permitem automatizar testes e atualizações do sistema, bem como manter versionamento do código e cobertura de testes [10];
- Criar uma interface responsiva com suporte a tema escuro e acessibilidade.

B. Justificativa

A necessidade de organizar tarefas não é nova, mas se intensificou com o crescimento do ensino remoto, projetos colaborativos e múltiplas responsabilidades simultâneas. Dados do Instituto McKinsey indicam que o uso de tecnologias colaborativas pode aumentar a produtividade em até 25% [16]. Entretanto, soluções amplamente utilizadas como o Microsoft To Do [17] e o Planner do Google Agenda [13], apesar de acessíveis, apresentam limitações quando se trata de funcionalidades mais avançadas, como registro de histórico de alterações, filtragem por múltiplos critérios, visualização por *status* com cores e notificações com horário programado.

O *NexTask* foi concebido justamente para preencher essa lacuna: uma ferramenta leve, focada, com funcionalidades pontuais pensadas para contextos acadêmicos e rotinas pessoais. A escolha por desenvolver esse sistema surgiu da observação de colegas e do próprio uso cotidiano de ferramentas de lista, que frequentemente deixavam a desejar em termos de notificações, histórico ou categorização flexível.

Além disso, o projeto possibilitou aplicar na prática conceitos de arquitetura web, segurança, testes de software e *design de interface*, promovendo uma experiência técnica completa e alinhada ao mercado. A adoção de metodologias ágeis também contribuiu para um desenvolvimento iterativo e com foco constante em entregas funcionais [21].

II. FUNDAMENTAÇÃO TEÓRICA

A Engenharia de Software é uma área da computação que busca aplicar metodologias, boas práticas e ferramentas para o desenvolvimento de sistemas de maneira estruturada, previsível e com qualidade garantida [23]. Segundo Pressman [21], a Engenharia de Software visa garantir que os

sistemas sejam construídos com confiabilidade, facilidade de manutenção e alinhamento aos requisitos dos usuários.

A. Testes de Software

Testes de software são atividades fundamentais no ciclo de vida de um sistema, com o objetivo de identificar falhas e assegurar que o software atenda aos requisitos esperados [3]. Dentre as principais abordagens, destacam-se os testes de caixa branca e os de caixa preta [5].

Testes de caixa branca analisam a estrutura interna do código, cobrindo caminhos lógicos, estruturas de decisão e fluxos internos [4]. Já os testes de caixa preta avaliam o comportamento do sistema com base nas entradas e saídas esperadas, sem considerar a lógica interna da implementação [23]. A combinação dessas abordagens permite uma verificação mais abrangente da aplicação, tanto no nível estrutural quanto funcional [18].

B. Usabilidade

A usabilidade é um atributo essencial em sistemas interativos, impactando diretamente na aceitação e satisfação do usuário [11]. De acordo com Nielsen [19], um sistema usável deve ser fácil de aprender, eficiente, memorável, com baixo índice de erros e satisfatório ao usuário.

Ao considerar esses critérios durante o desenvolvimento, é possível construir aplicações mais acessíveis e com melhor experiência de uso, especialmente importantes em contextos educacionais e de produtividade [1].

C. Controle de Versão e Integração Contínua

O controle de versão é uma prática que permite o registro e gerenciamento das modificações realizadas no código ao longo do tempo [7]. Essa abordagem é fundamental para projetos colaborativos, pois permite a rastreabilidade de alterações, facilita o trabalho em equipe e evita conflitos de edição [6].

A integração contínua, por sua vez, é uma técnica que automatiza o processo de validação de código sempre que alterações são inseridas no projeto [8]. Essa prática promove a detecção precoce de falhas, reduz o retrabalho e contribui para a entrega contínua de software com maior qualidade e estabilidade [21].

III. ABORDAGEM

O projeto *NexTask* caracteriza-se como uma pesquisa aplicada, com abordagem qualitativa e natureza prática, voltada à construção de uma aplicação web funcional, acessível e com foco em usabilidade. O desenvolvimento seguiu uma metodologia incremental, dividida em etapas: levantamento de requisitos, prototipagem, codificação, aplicação de testes e ajustes finais [21].

A. Arquitetura e Tecnologias Utilizadas

A arquitetura do sistema segue o modelo cliente-servidor [24]. No *front-end*, foram utilizadas tecnologias como

HTML5¹, CSS3², *Bootstrap* 5³ e *JavaScript* puro⁴ para garantir uma *interface* responsiva e de fácil navegação. O *back-end* foi construído com *Node.js*⁵ e *Express.js*⁶, utilizando o banco de dados *PostgreSQL*⁷ para armazenamento persistente das informações. Essa escolha reflete a adoção de tecnologias modernas, escaláveis e amplamente utilizadas no mercado atual, em contraste com abordagens mais tradicionais como PHP e MySQL, bastante comuns em gerações anteriores de aplicações web [25]. O controle de versões foi realizado com *Git*⁸, e o repositório foi hospedado no *GitHub*⁹, onde também foram gerenciadas *issues* e discutidas tarefas em equipe. O ambiente de desenvolvimento principal foi o *Visual Studio Code*¹⁰.

A autenticação dos usuários foi implementada com *tokens JWT (JSON Web Token)*, garantindo segurança no acesso às funcionalidades restritas e proteção de dados [9], [15]. Os dados do usuário autenticado são armazenados localmente no navegador para persistência da sessão [20].

B. Funcionalidades Principais

O sistema permite o gerenciamento completo de tarefas por meio das seguintes funcionalidades:

- Cadastro, edição, exclusão e listagem de tarefas;
- Aplicação de categorias e níveis de prioridade;
- Definição de *status* para cada tarefa: Pendente, Em andamento, Concluída e Atrasada;
- Identificação de *status* por cores: amarelo (pendente), azul (em andamento), verde (concluída) e vermelho (atrasada);
- Marcação de tarefa como concluída via *checkbox* diretamente no *card*;
- Aplicação de lembretes com data e hora definidos, com opção de alerta com 15 ou 30 minutos de antecedência;
- Exportação de tarefas em formato CSV;
- Filtro de tarefas por *status*, categoria, prioridade e data;
- Registro de comentários em tarefas durante o processo de edição, permitindo adicionar observações, instruções ou justificativas associadas a cada atividade;
- *Log* de atividades que registra todas as alterações realizadas em cada tarefa (edição, *status*, exclusão, etc.);
- Fixação de tarefas no topo da lista por meio de botão de estrela, facilitando o destaque de atividades importantes;
- Botão de "Limpar Concluídas", que remove todas as tarefas com *status* finalizado de forma rápida;
- Contador dinâmico de tarefas pendentes e em andamento exibido no título da aba do navegador.

C. Interface e Experiência do Usuário

A interface do *NexTask* foi pensada para ser simples, funcional e agradável. Os usuários podem alternar entre tema claro e tema escuro conforme preferência pessoal. Essa escolha é salva automaticamente no navegador, permitindo que o tema preferido seja mantido mesmo após o encerramento da sessão.

A Figura 2 apresenta a interface no tema escuro, com visual moderno e destaque de informações por meio de ícones e cores.

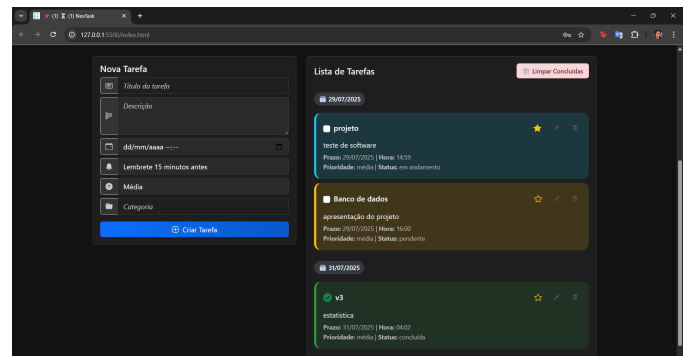


Figura 2. Tela principal no tema escuro com tarefas cadastradas

Já a Figura 3 mostra o sistema no tema claro, proporcionando legibilidade e contraste ideais para diferentes preferências de ambiente ou acessibilidade.

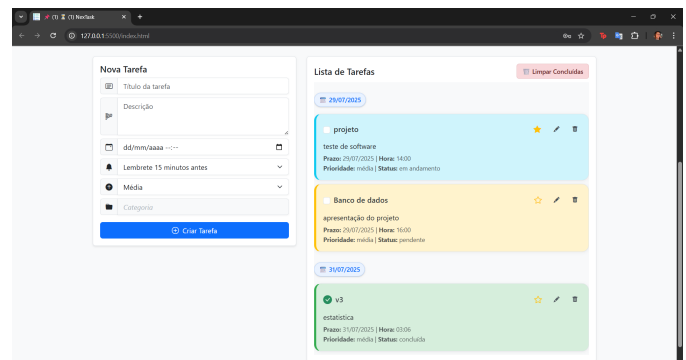


Figura 3. Tela principal no tema claro

Os *cards* de tarefas são organizados visualmente com uso de cores, ícones e marcações que facilitam a leitura e o entendimento rápido do *status* de cada atividade. Modais são utilizados para edição e visualização de detalhes das tarefas, evitando recargas de página e proporcionando fluidez na navegação.

A Figura 4 ilustra a mensagem amigável apresentada quando todas as tarefas foram concluídas, reforçando a proposta de uma interface acolhedora e motivacional.

¹<https://developer.mozilla.org/en-US/docs/Web/Guide/HTML/HTML5>

²<https://developer.mozilla.org/en-US/docs/Web/CSS>

³<https://getbootstrap.com/>

⁴<https://developer.mozilla.org/en-US/docs/Web/JavaScript>

⁵<https://nodejs.org/>

⁶<https://expressjs.com/>

⁷<https://www.postgresql.org/>

⁸<https://git-scm.com/>

⁹<https://github.com/angelosilvanno/gerenciadortarefas>

¹⁰<https://code.visualstudio.com/>

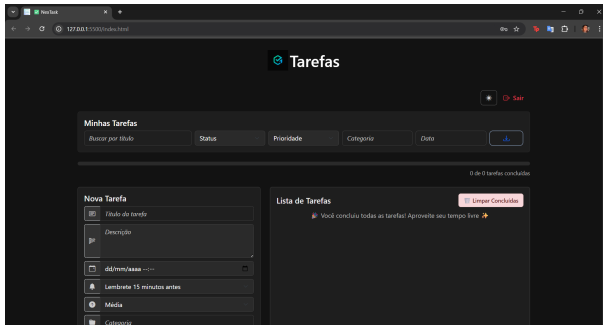


Figura 4. Mensagem exibida quando não há tarefas cadastradas, promovendo motivação e uma experiência positiva ao usuário

Durante o *login*, o sistema oferece a opção de exibir ou ocultar a senha digitada, aumentando a usabilidade em diferentes dispositivos e contextos de uso, especialmente para usuários que desejam verificar a senha antes de prosseguir com a autenticação.

No momento do cadastro, são solicitados o nome do usuário, *e-mail*, senha e confirmação de senha. O campo de nome de usuário exige entre 3 e 15 caracteres, podendo conter letras e/ou números. A senha deve ter entre 6 e 80 caracteres, contendo obrigatoriamente ao menos uma letra e um número, como forma de garantir um nível mínimo de segurança e evitar cadastros fracos ou inconsistentes.

O sistema também verifica se o *e-mail* informado já está cadastrado. Caso o usuário tente registrar um *e-mail* que já existe na base de dados, uma mensagem de aviso é exibida, impedindo o cadastro duplicado e reforçando o controle de integridade e a experiência do usuário.

D. Testes Realizados

Foram aplicados testes de software com abordagem de caixa branca (testes unitários e de fluxo interno do código) e caixa preta (testes de comportamento esperado a partir das entradas). Os testes foram automatizados sempre que possível, utilizando o *framework Jest* para lógica *JavaScript*, e testes manuais com base em cenários de uso reais. Além disso, a cobertura de testes foi verificada e incluída no repositório para controle da qualidade.

E. Integração Contínua e Boas Práticas

A integração contínua foi implementada com *GitHub Actions*, permitindo a execução automática dos testes unitários sempre que um novo *commit* é realizado. Isso garantiu que falhas fossem identificadas precocemente e corrigidas de forma ágil. Além disso, o projeto foi conduzido com divisão de responsabilidades entre os integrantes mais engajados, organização por *branches*, revisão de código e foco em entregas iterativas, seguindo boas práticas de Engenharia de Software moderna.

IV. CONSIDERAÇÕES FINAIS E TRABALHOS FUTUROS

O desenvolvimento do sistema *NexTask* permitiu consolidar, na prática, diversos conceitos de Engenharia de Software, com ênfase em usabilidade, testes de qualidade e organização

de tarefas em ambiente web. Por meio de uma abordagem incremental, foram aplicadas técnicas modernas de desenvolvimento, integração contínua, versionamento e testes automatizados.

O sistema entregue apresenta uma interface intuitiva, suporte a tema claro e escuro com persistência, lembretes programáveis, categorização por prioridade, comentários em tarefas, *log* de alterações, filtros inteligentes, fixação de tarefas prioritárias no topo, botão para limpar tarefas concluídas e contador dinâmico de tarefas pendentes e em andamento na aba do navegador. A aplicação demonstrou estabilidade e bom desempenho, passando com sucesso pelos testes de caixa branca e preta, cobrindo os principais fluxos e validações do sistema.

Os testes automatizados alcançaram **100% de cobertura de instruções, funções e linhas de código, e 95,45% de cobertura de *branches***, conforme relatório gerado com a ferramenta *Istanbul*¹¹, mostrado na Figura 5, o que reforça a robustez e qualidade da aplicação.

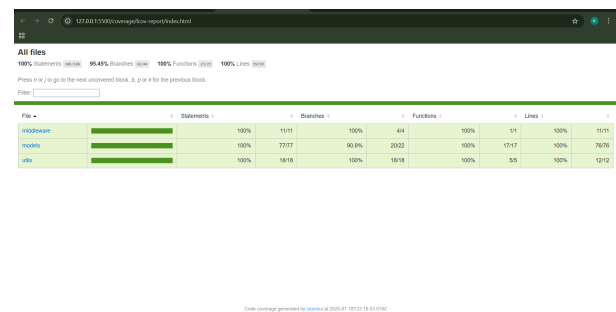


Figura 5. Relatório de cobertura de testes automatizados gerado com a ferramenta Istanbul

Além disso, a atenção aos detalhes - como validação de senha e nome de usuário, exibição de senha no *login*, controle de *e-mails* duplicados e diferenciação visual por *status* de tarefas - evidencia o cuidado técnico aplicado ao longo do projeto. A exportação em CSV, o uso de *tokens JWT* e a estrutura modular do código contribuíram para a segurança, rastreabilidade e manutenibilidade do sistema.

A Figura 6 ilustra o *log* de alterações presente na aplicação, funcionalidade que registra e exibe com clareza o histórico de ações realizadas em cada tarefa - como edições, conclusões ou exclusões -, reforçando a transparência e rastreabilidade no gerenciamento.

¹¹<https://istanbul.js.org/>

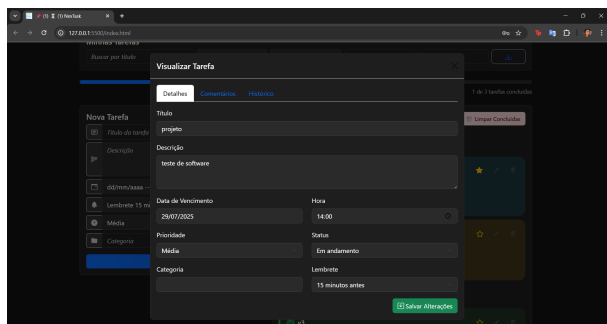


Figura 6. Visualização do log de atividades com histórico de alterações nas tarefas

O sistema encontra-se atualmente publicado e acessível ao público por meio do link: nexttask-app.onrender.com. Essa disponibilização visa facilitar a demonstração da aplicação e promover testes reais da solução em diferentes dispositivos e contextos de uso.

A. Trabalhos Futuros

Como possíveis aprimoramentos futuros, destacam-se:

- Implementação de notificações em tempo real por *email* ou via *push*;
- Melhorias na função de notificações;
- Adição de suporte para anexos em tarefas (ex: imagens, documentos);
- Integração com calendário externo (como Google Agenda);
- Criação de relatórios gráficos com estatísticas de produtividade;
- Expansão da base de usuários com diferentes níveis de permissão (*admin*, colaborador, convidado);
- Recuperação de senha por *email*, funcionalidade que chegou a ser iniciada, mas precisou ser retirada da versão final por limitações de tempo.

O sistema encontra-se atualmente publicado e acessível ao público por meio do link: <https://nexttask-app.onrender.com/>. Essa disponibilização visa facilitar a demonstração da aplicação e promover testes reais da solução em diferentes dispositivos e contextos de uso.

REFERÊNCIAS

- [1] ALAM, M. S., ULLAH, M. H., AND HASAN, M. M. Impact of user interface on user experience in e-learning platforms. *International Journal of Computer Applications* 176, 39 (2020), 10–15.
- [2] ALVES, G. A., AND OLIVEIRA, J. F. D. Organização do tempo e produtividade acadêmica: desafios enfrentados pelos universitários. *Revista Ibero-Americana de Estudos em Educação* 16, 1 (2021), 202–219.
- [3] AMMANN, P., AND OFFUTT, J. *Introduction to Software Testing*, 2 ed. Cambridge University Press, 2016.
- [4] AMMANN, P., AND OFFUTT, J. *Introduction to Software Testing*, 2 ed. Cambridge University Press, 2016.
- [5] BEIZER, B. *Software Testing Techniques*, 2 ed. Dreamtech Press, 1995.
- [6] BIRD, C., NAGAPPAN, N., GALL, H., MURPHY, B., AND DEVANBU, P. Promises and perils in the adoption of developer testing: a case study. In *Proceedings of the 2009 6th IEEE International Working Conference on Mining Software Repositories* (2009), IEEE, pp. 97–106.
- [7] CHACON, S., AND STRAUB, B. *Pro Git*, 2 ed. Apress, 2014.
- [8] DUVALL, P. M., MATYAS, S., AND GLOVER, A. *Continuous Integration: Improving Software Quality and Reducing Risk*. Addison-Wesley Professional, 2006.
- [9] FERREIRA, D. Como funciona o jwt (json web token). <https://medium.com/trainingcenter/como-funciona-o-jwt-json-web-token-4b43f483f552>, 2018. Acessado em julho de 2025.
- [10] FOWLER, M. *Continuous Integration: Improving Software Quality and Reducing Risk*. Addison-Wesley Professional, 2018.
- [11] GARRETT, J. J. *The Elements of User Experience: User-Centered Design for the Web and Beyond*. New Riders, 2010.
- [12] GITHUB. About continuous integration with github actions. <https://docs.github.com/en/actions/automating-builds-and-tests/about-continuous-integration>, 2024. Accessed: 2025-07-24.
- [13] GOOGLE LLC. Google Calendar – Planner. <https://calendar.google.com/>, 2024. Acesso em: jul. 2025.
- [14] JONES, M. B., BRADLEY, J., AND SAKIMURA, N. Json web token (jwt). Tech. Rep. RFC 7519, Internet Engineering Task Force (IETF), 2015.
- [15] JOSEPH, V. *Mastering JWT: Secure Your Applications with JSON Web Tokens*. Independently published, 2022.
- [16] MANYIKA, J., CHUI, M., BUGHIN, J., DOBBS, R., ROXBURGH, C., AND BYERS, A. H. The social economy: Unlocking value and productivity through social technologies. *McKinsey Global Institute* (2012).
- [17] MICROSOFT CORPORATION. Microsoft To Do. <https://todo.microsoft.com/>, 2024. Acesso em: jul. 2025.
- [18] MYERS, G. J., SANDLER, C., AND BADGETT, T. *The Art of Software Testing*, 3 ed. John Wiley & Sons, 2011.
- [19] NIELSEN, J. *Usability Engineering*. Morgan Kaufmann, 1994.
- [20] OWASP FOUNDATION. Local storage security guide, 2023. Acesso em: 25 jul. 2025.
- [21] PRESSMAN, R. S., AND MAXIM, B. R. *Engenharia de Software*, 8 ed. McGraw Hill Brasil, 2016.
- [22] PÉREZ, A., AND PÉREZ, L. Digital tools for collaborative learning in higher education: systematic review and future research agenda. *International Journal of Educational Technology in Higher Education* 17, 1 (2020), 1–26.
- [23] SOMMERVILLE, I. *Software Engineering*, 9 ed. Pearson Education, 2011.
- [24] TANENBAUM, A. S., AND STEEN, M. V. *Sistemas Distribuídos: Princípios e Paradigmas*, 2 ed. Pearson Prentice Hall, 2008.
- [25] WELLING, L., AND THOMSON, L. *PHP and MySQL Web Development*. Addison-Wesley, 2003.